DSA - 50 Questions by Pattern

Brute force and optimised, with time and space complexity

Nirmalya Mandal - SAP Labs Interview Prep

Study pack - part 5 of 8

Contents

How to use this

The fastest way to get good at DSA quickly is to learn **patterns**, not memorise 500 problems. Almost every interview question is one of about a dozen patterns wearing a costume. This document gives you **50 questions grouped by pattern**, mostly easy and medium with a few harder ones, and for each you get:

- the **problem** in one line,
- the **brute force** approach with its **time and space complexity**,
- the **optimised** approach with C++ code and its **time and space complexity**,
- and the **pattern takeaway** so you recognise it next time.

How to study this: don't read passively. For each pattern, understand *why* the optimised trick works, then try to solve the next question in that pattern yourself before reading the solution. When you can look at a new problem and say "this is a sliding-window problem," you've won.

Notation: `n` is the input size. Complexities are written as time / space.

A quick key to the recurring optimisation ideas:

- **Hashing** turns an O(n) search inside a loop into O(1), collapsing O(n^2) to O(n).
- **Two pointers** replaces a nested loop with two indices moving inward or forward.
- **Sliding window** avoids recomputing an overlapping range from scratch.
- **Sorting first** unlocks two-pointer and greedy tricks (costs O(n log n)).
- **Binary search** halves the space when data is sorted or the answer is monotonic.
- **DP** stores sub-answers so you never recompute them.

Pattern 1: Two Pointers

Use two indices moving toward each other (sorted array) or in the same direction. Turns many O(n^2) scans into O(n).

Q1. Two Sum II - input is sorted (Easy)

Problem: given a **sorted** array and a target, return indices of two numbers that add to the target.

Brute force: check every pair. **O(n^2) / O(1).**

Optimised (two pointers): one pointer at each end; if the sum is too big move right pointer left, if too small move left pointer right.

```
vector<int> twoSum(vector<int>& a, int target) {
    int l = 0, r = a.size() - 1;
    while (l < r) {
        int s = a[l] + a[r];
        if (s == target) return {l, r};
        if (s < target) l++;
        else r--;
    }
    return {};
}
```

O(n) / O(1). *Takeaway:* sorted + "find a pair" -> two pointers from the ends.

Q2. Valid Palindrome (Easy)

Problem: is a string a palindrome, ignoring non-alphanumerics and case?

Brute force: build a cleaned string, reverse it, compare. **O(n) / O(n).**

Optimised (two pointers): compare from both ends inward, no extra string.

```
bool isPalindrome(string s) {
    int l = 0, r = s.size() - 1;
    while (l < r) {
        if (!isalnum(s[l])) { l++; continue; }
        if (!isalnum(s[r])) { r--; continue; }
        if (tolower(s[l]) != tolower(s[r])) return false;
        l++; r--;
    }
    return true;
}
```

O(n) / O(1). *Takeaway:* palindrome checks are two pointers from the ends.

Q3. Remove Duplicates from Sorted Array (Easy)

Problem: remove duplicates in place from a sorted array, return new length.

Brute force: copy uniques into a new array/set. **O(n) / O(n).**

Optimised (slow/fast pointers): a slow pointer marks the write position; fast scans ahead.

```
int removeDuplicates(vector<int>& a) {
    if (a.empty()) return 0;
    int slow = 0;
    for (int fast = 1; fast < a.size(); fast++) {
        if (a[fast] != a[slow]) {
            slow++;
            a[slow] = a[fast];
        }
    }
    return slow + 1;
}
```

O(n) / O(1). *Takeaway:* in-place array edits use a slow write pointer + fast read pointer.

Q4. Container With Most Water (Medium)

Problem: given heights, pick two lines forming the container holding the most water.

Brute force: try all pairs, area = min(h[i], h[j]) * (j - i). **O(n^2) / O(1).**

Optimised (two pointers): start wide; area is limited by the shorter line, so move the shorter pointer inward hoping for a taller one.

```
int maxArea(vector<int>& h) {
    int l = 0, r = h.size() - 1, best = 0;
    while (l < r) {
        int area = min(h[l], h[r]) * (r - l);
        best = max(best, area);
        if (h[l] < h[r]) l++;
        else r--;
    }
    return best;
}
```

O(n) / O(1). *Takeaway:* move the pointer that limits you; width shrinks, so height must grow.

Q5. 3Sum (Medium-Hard)

Problem: find all unique triplets that sum to zero.

Brute force: three nested loops + a set to dedupe. **O(n^3) / O(n).**

Optimised (sort + two pointers): fix one number, two-pointer the rest; skip duplicates.

```
vector<vector<int>> threeSum(vector<int>& a) {
    sort(a.begin(), a.end());
    vector<vector<int>> res;
    int n = a.size();
    for (int i = 0; i < n - 2; i++) {
        if (i > 0 && a[i] == a[i - 1]) continue;   // skip dup
        int l = i + 1, r = n - 1;
        while (l < r) {
            int s = a[i] + a[l] + a[r];
            if (s == 0) {
                res.push_back({a[i], a[l], a[r]});
                l++; r--;
                while (l < r && a[l] == a[l - 1]) l++;
                while (l < r && a[r] == a[r + 1]) r--;
            } else if (s < 0) l++;
            else r--;
        }
    }
    return res;
}
```

O(n^2) / O(1) extra (ignoring output). *Takeaway:* sort, then reduce a K-sum to fixing one element and two-pointering the rest.

Pattern 2: Sliding Window

For problems about a **contiguous subarray/substring** of a range or size. Grow/shrink a window instead of recomputing it from scratch.

Q6. Maximum Sum Subarray of Size K (Easy)

Problem: largest sum of any k consecutive elements.

Brute force: sum every window of size k. **O(n*k) / O(1).**

Optimised (fixed window): slide by adding the new element and removing the one that left.

```
int maxSumK(vector<int>& a, int k) {
    int sum = 0;
    for (int i = 0; i < k; i++) sum += a[i];
    int best = sum;
    for (int i = k; i < a.size(); i++) {
        sum += a[i] - a[i - k];   // add new, drop old
        best = max(best, sum);
    }
    return best;
}
```

O(n) / O(1). *Takeaway:* fixed-size window updates in O(1) per step.

Q7. Longest Substring Without Repeating Characters (Medium)

Problem: length of the longest substring with all unique characters.

Brute force: check every substring for uniqueness. **O(n^2) or O(n^3) / O(n).**

Optimised (variable window + hashset): expand right; when a duplicate appears, shrink from left.

```
int lengthOfLongest(string s) {
    unordered_set<char> seen;
    int l = 0, best = 0;
    for (int r = 0; r < s.size(); r++) {
        while (seen.count(s[r])) {
            seen.erase(s[l]);
            l++;
        }
        seen.insert(s[r]);
        best = max(best, r - l + 1);
    }
    return best;
}
```

O(n) / O(min(n, charset)). *Takeaway:* variable window - grow right, shrink left when a rule breaks.

Q8. Minimum Size Subarray Sum (Medium)

Problem: smallest length of a contiguous subarray with sum >= target (positives).

Brute force: try all subarrays. **O(n^2) / O(1).**

Optimised (shrinking window): expand right adding to sum; while sum >= target, record length and shrink left.

```
int minSubArrayLen(int target, vector<int>& a) {
    int l = 0, sum = 0, best = INT_MAX;
    for (int r = 0; r < a.size(); r++) {
        sum += a[r];
        while (sum >= target) {
            best = min(best, r - l + 1);
            sum -= a[l];
            l++;
        }
    }
    return best == INT_MAX ? 0 : best;
}
```

O(n) / O(1). *Takeaway:* "shortest/longest subarray with a sum condition" -> sliding window.

Q9. Longest Repeating Character Replacement (Hard-ish)

Problem: longest substring you can make all-same by replacing at most k characters.

Brute force: for every window, count the majority char and check replacements needed. **O(n^2) / O(1).**

Optimised (window + max frequency): a window is valid if `(windowLen - maxFreq) <= k`.

```
int characterReplacement(string s, int k) {
    vector<int> cnt(26, 0);
    int l = 0, maxFreq = 0, best = 0;
    for (int r = 0; r < s.size(); r++) {
        cnt[s[r] - 'A']++;
        maxFreq = max(maxFreq, cnt[s[r] - 'A']);
        while ((r - l + 1) - maxFreq > k) {
            cnt[s[l] - 'A']--;
            l++;
        }
        best = max(best, r - l + 1);
    }
    return best;
}
```

O(n) / O(1). *Takeaway:* window validity can depend on a running max frequency, not just a sum.

Pattern 3: Hashing / Hash Map

When you need fast "have I seen this?" or "how many of this?", a hash map turns an inner O(n) search into O(1).

Q10. Two Sum (Easy)

Problem: indices of two numbers adding to a target (unsorted).

Brute force: all pairs. **O(n^2) / O(1).**

Optimised (hash map): store each number's index; look up `target - a[i]`.

```
vector<int> twoSum(vector<int>& a, int target) {
    unordered_map<int, int> seen;
    for (int i = 0; i < a.size(); i++) {
        int need = target - a[i];
        if (seen.count(need)) return {seen[need], i};
        seen[a[i]] = i;
    }
    return {};
}
```

O(n) / O(n). *Takeaway:* "find a complement" -> store seen values in a hash map.

Q11. Contains Duplicate (Easy)

Problem: does any value appear twice?

Brute force: compare all pairs. **O(n^2) / O(1).**

Optimised (hash set): insert into a set; if already present, duplicate found.

```
bool containsDuplicate(vector<int>& a) {
    unordered_set<int> seen;
    for (int x : a) {
        if (seen.count(x)) return true;
        seen.insert(x);
    }
    return false;
}
```

O(n) / O(n). *Takeaway:* uniqueness checks -> hash set.

Q12. Valid Anagram (Easy)

Problem: is string t a rearrangement of string s?

Brute force: sort both and compare. **O(n log n) / O(1).**

Optimised (frequency count): count letters of s, subtract with t; all zero means anagram.

```cpp
bool isAnagram(string s, string t) {
    if (s.size() != t.size()) return false;
    vector<int> cnt(26, 0);
    for (char c : s) cnt[c - 'a']++;
    for (char c : t) {
        if (--cnt[c - 'a'] < 0) return false;
    }
    return true;
}
```

O(n) / O(1). *Takeaway:* anagram/frequency problems -> a count array.

Q13. Group Anagrams (Medium)

Problem: group words that are anagrams of each other.

Brute force: compare every pair for anagram-ness. **O(n^2 * k) / O(1).**

Optimised (hash map keyed by sorted word): words sharing a sorted form are anagrams.

```cpp
vector<vector<string>> groupAnagrams(vector<string>& v) {
    unordered_map<string, vector<string>> m;
    for (string& w : v) {
        string key = w;
        sort(key.begin(), key.end());
        m[key].push_back(w);
    }
    vector<vector<string>> res;
    for (auto& p : m) res.push_back(p.second);
    return res;
}
```

O(n * k log k) / O(n*k). *Takeaway:* group-by problems -> build a hash map with a canonical key.

Pattern 4: Prefix Sum

Precompute cumulative sums so any range sum is O(1). Great for subarray-sum questions.

Q14. Range Sum Query - Immutable (Easy)

Problem: answer many "sum from i to j" queries on a fixed array.

Brute force: sum i..j each query. **O(n) per query.**

Optimised (prefix array): `pre[i]` = sum of first i elements; range = `pre[j+1] - pre[i]`.

```
vector<int> pre;
void build(vector<int>& a) {
    pre.assign(a.size() + 1, 0);
    for (int i = 0; i < a.size(); i++)
        pre[i + 1] = pre[i] + a[i];
}
int rangeSum(int i, int j) { return pre[j + 1] - pre[i]; }
```

Build O(n); each query O(1) / O(n). *Takeaway:* many range-sum queries -> prefix sums.

Q15. Subarray Sum Equals K (Medium)

Problem: count contiguous subarrays that sum to k.

Brute force: sum every subarray. **O(n^2) / O(1).**

Optimised (prefix sum + hash map): if `prefix - k` was seen before, those subarrays sum to k.

```
int subarraySum(vector<int>& a, int k) {
    unordered_map<int, int> cnt;
    cnt[0] = 1;
    int prefix = 0, ans = 0;
    for (int x : a) {
        prefix += x;
        ans += cnt[prefix - k];
        cnt[prefix]++;
    }
    return ans;
}
```

O(n) / O(n). *Takeaway:* "count subarrays with sum k" -> prefix sums stored in a hash map.

Q16. Find Pivot Index (Easy)

Problem: index where the left sum equals the right sum.

Brute force: for each i, sum left and right. **O(n^2) / O(1).**

Optimised (total - prefix): right sum = total - prefix - a[i].

```cpp
int pivotIndex(vector<int>& a) {
    int total = 0, left = 0;
    for (int x : a) total += x;
    for (int i = 0; i < a.size(); i++) {
        if (left == total - left - a[i]) return i;
        left += a[i];
    }
    return -1;
}
```

O(n) / O(1). *Takeaway:* balance-point problems -> track running left sum vs total.

Pattern 5: Binary Search

Sorted data or a monotonic answer space -> halve it each step. O(log n).

Q17. Binary Search (Easy)

Problem: find a target in a sorted array. *Brute force:* linear scan, **O(n).** *Optimised:* halve each step.

```cpp
int search(vector<int>& a, int target) {
    int lo = 0, hi = a.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == target) return mid;
        if (a[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

O(log n) / O(1). *Takeaway:* sorted + search = binary search.

Q18. First and Last Position of a Target (Medium)

Problem: first and last index of a target in a sorted array with duplicates.

Brute force: linear scan tracking first/last. **O(n) / O(1).**

Optimised: two binary searches - one biased left, one biased right.

```cpp
int bound(vector<int>& a, int t, bool first) {
    int lo = 0, hi = a.size() - 1, res = -1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == t) {
            res = mid;
            if (first) hi = mid - 1;    // keep going left
            else lo = mid + 1;          // keep going right
        } else if (a[mid] < t) lo = mid + 1;
        else hi = mid - 1;
    }
    return res;
}
```

O(log n) / O(1). *Takeaway:* "first/last occurrence" -> binary search that keeps moving past a match.

Q19. Search in Rotated Sorted Array (Medium)

Problem: search in a sorted array that has been rotated.

Brute force: linear scan. **O(n) / O(1).**

Optimised: one half is always sorted; decide which, and whether the target is in it.

```
int search(vector<int>& a, int t) {
    int lo = 0, hi = a.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == t) return mid;
        if (a[lo] <= a[mid]) {              // left sorted
            if (a[lo] <= t && t < a[mid]) hi = mid - 1;
            else lo = mid + 1;
        } else {                            // right sorted
            if (a[mid] < t && t <= a[hi]) lo = mid + 1;
            else hi = mid - 1;
        }
    }
    return -1;
}
```

O(log n) / O(1). *Takeaway:* rotated array - find the sorted half, then decide.

Q20. Find Minimum in Rotated Sorted Array (Medium)

Problem: find the smallest element in a rotated sorted array.

Brute force: linear min. **O(n).** *Optimised:* binary search toward the unsorted side.

```
int findMin(vector<int>& a) {
    int lo = 0, hi = a.size() - 1;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] > a[hi]) lo = mid + 1;  // min is right
        else hi = mid;                     // min is mid or left
    }
    return a[lo];
}
```

O(log n) / O(1). *Takeaway:* compare mid to the ends to find which side holds the pivot.

Q21. Koko Eating Bananas (Medium - binary search on the answer)

Problem: smallest eating speed to finish all piles within h hours.

Brute force: try every speed from 1 upward. **O(maxPile * n).**

Optimised: the answer is monotonic (faster speed always finishes in fewer hours), so binary-search the speed.

```cpp
long long hours(vector<int>& p, int speed) {
    long long h = 0;
    for (int x : p) h += (x + speed - 1) / speed;  // ceil
    return h;
}
int minEatingSpeed(vector<int>& p, int H) {
    int lo = 1, hi = *max_element(p.begin(), p.end());
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (hours(p, mid) <= H) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
```

O(n log maxPile) / O(1). *Takeaway:* if "is X feasible?" is monotonic, binary-search X. This is a very common trick.

Pattern 6: Fast & Slow Pointers / Linked List

Pointer manipulation, and the two-speed-pointer trick for cycles and midpoints.

Q22. Reverse a Linked List (Easy)

Problem: reverse a singly linked list.

Brute force: push values to an array and rebuild. **O(n) / O(n).**

Optimised (iterative pointer flip): reverse pointers in one pass.

```
ListNode* reverse(ListNode* head) {
    ListNode* prev = nullptr;
    while (head) {
        ListNode* nxt = head->next;
        head->next = prev;
        prev = head;
        head = nxt;
    }
    return prev;
}
```

O(n) / O(1). *Takeaway:* keep prev/cur/next and flip links one at a time.

Q23. Linked List Cycle (Easy)

Problem: does the list have a cycle?

Brute force: store visited nodes in a set. **O(n) / O(n).**

Optimised (Floyd's fast/slow): if a fast pointer laps a slow one, there's a cycle.

```
bool hasCycle(ListNode* head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}
```

O(n) / O(1). *Takeaway:* cycle detection = two pointers at different speeds.

Q24. Middle of the Linked List (Easy)

Problem: return the middle node.

Brute force: count length, then walk to n/2. **O(n) / O(1), two passes.**

Optimised: fast moves twice as fast; when it ends, slow is at the middle (one pass).

```
ListNode* middleNode(ListNode* head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}
```

O(n) / O(1). *Takeaway:* fast/slow finds the midpoint in one pass.

Q25. Merge Two Sorted Lists (Easy)

Problem: merge two sorted lists into one sorted list.

Optimised (dummy head): attach the smaller head each step.

```
ListNode* mergeTwoLists(ListNode* a, ListNode* b) {
    ListNode dummy(0), *tail = &dummy;
    while (a && b) {
        if (a->val <= b->val) { tail->next = a; a = a->next; }
        else { tail->next = b; b = b->next; }
        tail = tail->next;
    }
    tail->next = a ? a : b;
    return dummy.next;
}
```

O(n + m) / O(1). *Takeaway:* a dummy head removes annoying empty-list edge cases.

Q26. Remove Nth Node From End (Medium)

Problem: remove the nth node counting from the end, in one pass.

Optimised (gap of n between two pointers): advance fast by n, then move both until fast ends.

```
ListNode* removeNthFromEnd(ListNode* head, int n) {
    ListNode dummy(0); dummy.next = head;
    ListNode *fast = &dummy, *slow = &dummy;
    for (int i = 0; i < n; i++) fast = fast->next;
    while (fast->next) { fast = fast->next; slow = slow->next; }
    slow->next = slow->next->next;   // skip the node
    return dummy.next;
}
```

O(n) / O(1). *Takeaway:* "nth from the end" -> keep two pointers n apart.

Pattern 7: Stack

Use a stack for "match the most recent thing" or "nearest greater/smaller" (monotonic stack).

Q27. Valid Parentheses (Easy)

Problem: is a string of brackets balanced and correctly nested?

Optimised (stack): push openers; on a closer, the top must be its match.

```
bool isValid(string s) {
    stack<char> st;
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') st.push(c);
        else {
            if (st.empty()) return false;
            char t = st.top(); st.pop();
            if ((c == ')' && t != '(') ||
                (c == ']' && t != '[') ||
                (c == '}' && t != '{')) return false;
        }
    }
    return st.empty();
}
```

O(n) / O(n). *Takeaway:* nesting/matching problems -> stack.

Q28. Min Stack (Medium)

Problem: a stack that also returns its minimum in O(1).

Optimised: keep a second stack of running minimums.

```
stack<int> s, mn;
void push(int x) {
    s.push(x);
    if (mn.empty() || x <= mn.top()) mn.push(x);
    else mn.push(mn.top());
}
void pop() { s.pop(); mn.pop(); }
int top() { return s.top(); }
int getMin() { return mn.top(); }
```

All O(1) / O(n). *Takeaway:* track an auxiliary value alongside the main stack.

Q29. Daily Temperatures (Medium - monotonic stack)

Problem: for each day, how many days until a warmer temperature.

Brute force: for each day scan forward. **O(n^2) / O(1).**

Optimised (monotonic stack of indices): pop while the current day is warmer.

```
vector<int> dailyTemperatures(vector<int>& t) {
    vector<int> res(t.size(), 0);
    stack<int> st;                        // indices
    for (int i = 0; i < t.size(); i++) {
        while (!st.empty() && t[i] > t[st.top()]) {
            int j = st.top(); st.pop();
            res[j] = i - j;
        }
        st.push(i);
    }
    return res;
}
```

O(n) / O(n). *Takeaway:* "next greater/smaller element" -> monotonic stack.

Q30. Evaluate Reverse Polish Notation (Medium)

Problem: evaluate an expression in postfix form (e.g. `["2","1","+","3","*"]`).

Optimised (stack): push numbers; on an operator, pop two and apply.

```
int evalRPN(vector<string>& tokens) {
    stack<int> st;
    for (string& tk : tokens) {
        if (tk == "+" || tk == "-" || tk == "*" || tk == "/") {
            int b = st.top(); st.pop();
            int a = st.top(); st.pop();
            if (tk == "+") st.push(a + b);
            else if (tk == "-") st.push(a - b);
            else if (tk == "*") st.push(a * b);
            else st.push(a / b);
        } else st.push(stoi(tk));
    }
    return st.top();
}
```

O(n) / O(n). *Takeaway:* postfix evaluation is the textbook stack use (relates to your CONTEXT interpreter).

Pattern 8: Trees (BFS / DFS)

Recursion is natural for trees. BFS uses a queue (level order); DFS uses recursion.

Q31. Maximum Depth of Binary Tree (Easy)

Optimised (DFS recursion): depth = 1 + max(left, right).

```
int maxDepth(TreeNode* root) {
    if (!root) return 0;
    return 1 + max(maxDepth(root->left), maxDepth(root->right));
}
```

O(n) / O(h) where h is height (recursion stack). *Takeaway:* tree size/height = simple DFS.

Q32. Invert Binary Tree (Easy)

Optimised (DFS): swap children, recurse.

```
TreeNode* invertTree(TreeNode* root) {
    if (!root) return nullptr;
    swap(root->left, root->right);
    invertTree(root->left);
    invertTree(root->right);
    return root;
}
```

O(n) / O(h). *Takeaway:* structural changes recurse down and combine up.

Q33. Same Tree (Easy)

Optimised (DFS): both null = same; values and both subtrees must match.

```
bool isSameTree(TreeNode* p, TreeNode* q) {
    if (!p && !q) return true;
    if (!p || !q || p->val != q->val) return false;
    return isSameTree(p->left, q->left) &&
           isSameTree(p->right, q->right);
}
```

O(n) / O(h). *Takeaway:* compare two trees by recursing in lockstep.

Q34. Binary Tree Level Order Traversal (Medium)

Problem: return node values level by level.

Optimised (BFS with a queue): process one whole level at a time.

```
vector<vector<int>> levelOrder(TreeNode* root) {
    vector<vector<int>> res;
    if (!root) return res;
    queue<TreeNode*> q;
    q.push(root);
    while (!q.empty()) {
        int sz = q.size();
        vector<int> level;
        for (int i = 0; i < sz; i++) {
            TreeNode* n = q.front(); q.pop();
            level.push_back(n->val);
            if (n->left) q.push(n->left);
            if (n->right) q.push(n->right);
        }
        res.push_back(level);
    }
    return res;
}
```

O(n) / O(n). *Takeaway:* "level by level" -> BFS, capturing the queue size per level.

Q35. Validate Binary Search Tree (Medium)

Problem: is a tree a valid BST?

Brute force: for each node check all descendants. **O(n^2).**

Optimised (range check): pass down a valid (min, max) range.

```
bool valid(TreeNode* n, long lo, long hi) {
    if (!n) return true;
    if (n->val <= lo || n->val >= hi) return false;
    return valid(n->left, lo, n->val) &&
           valid(n->right, n->val, hi);
}
bool isValidBST(TreeNode* root) {
    return valid(root, LONG_MIN, LONG_MAX);
}
```

O(n) / O(h). *Takeaway:* BST validity = each node must fit an inherited range.

Q36. Lowest Common Ancestor of a BST (Medium)

Problem: find the lowest common ancestor of two nodes in a BST.

Optimised (use BST order): if both are smaller go left, both bigger go right, else this is the split point.

```cpp
TreeNode* lca(TreeNode* root, TreeNode* p, TreeNode* q) {
    while (root) {
        if (p->val < root->val && q->val < root->val)
            root = root->left;
        else if (p->val > root->val && q->val > root->val)
            root = root->right;
        else return root;
    }
    return nullptr;
}
```

O(h) / O(1). *Takeaway:* in a BST, the ordering tells you which way to walk.

Pattern 9: Backtracking

Build candidates step by step; when you hit a dead end or a complete answer, undo the last choice and try another. Used for subsets, permutations, combinations.

Q37. Subsets (Medium)

Problem: all subsets (the power set) of a set of distinct numbers.

Optimised (backtracking): for each index, choose to include it or not.

```
void backtrack(vector<int>& a, int i, vector<int>& cur,
               vector<vector<int>>& res) {
    if (i == a.size()) { res.push_back(cur); return; }
    cur.push_back(a[i]);            // include a[i]
    backtrack(a, i + 1, cur, res);
    cur.pop_back();                 // exclude a[i]
    backtrack(a, i + 1, cur, res);
}
```

O(2^n * n) / O(n) depth. *Takeaway:* subsets = include/exclude each element.

Q38. Permutations (Medium)

Problem: all orderings of distinct numbers.

Optimised (backtracking with used[]): pick each unused number in turn.

```
void backtrack(vector<int>& a, vector<bool>& used,
               vector<int>& cur, vector<vector<int>>& res) {
    if (cur.size() == a.size()) { res.push_back(cur); return; }
    for (int i = 0; i < a.size(); i++) {
        if (used[i]) continue;
        used[i] = true; cur.push_back(a[i]);
        backtrack(a, used, cur, res);
        used[i] = false; cur.pop_back();   // undo
    }
}
```

O(n! * n) / O(n). *Takeaway:* permutations = pick each unused element, then undo.

Q39. Combination Sum (Medium)

Problem: all combinations (reuse allowed) that sum to a target.

Optimised (backtracking, stay at i to allow reuse):

```
void backtrack(vector<int>& a, int start, int target,
               vector<int>& cur, vector<vector<int>>& res) {
```

```
    if (target == 0) { res.push_back(cur); return; }
    if (target < 0) return;
    for (int i = start; i < a.size(); i++) {
        cur.push_back(a[i]);
        backtrack(a, i, target - a[i], cur, res);  // i, reuse
        cur.pop_back();
    }
}
```

Exponential / O(target) depth. *Takeaway:* a `start` index avoids duplicate combinations; reuse = recurse with the same i.

Q40. Generate Parentheses (Medium)

Problem: all valid combinations of n pairs of parentheses.

Optimised (backtracking with counts): add `(` if any left; add `)` only if it stays valid.

```
void backtrack(int open, int close, int n, string cur,
               vector<string>& res) {
    if (cur.size() == 2 * n) { res.push_back(cur); return; }
    if (open < n) backtrack(open + 1, close, n, cur + "(", res);
    if (close < open) backtrack(open, close + 1, n, cur + ")", res);
}
```

O(4^n / sqrt(n)) / O(n). *Takeaway:* prune invalid branches early with simple counts.

Pattern 10: Dynamic Programming

When a problem breaks into overlapping sub-problems, store sub-answers so you never recompute. Start by finding the recurrence.

Q41. Climbing Stairs (Easy)

Problem: ways to climb n stairs taking 1 or 2 steps.

Brute force: recurse `f(n)=f(n-1)+f(n-2)` - exponential.

Optimised (bottom-up, two variables): it's just Fibonacci.

```
int climbStairs(int n) {
    int a = 1, b = 1;
    for (int i = 2; i <= n; i++) {
        int c = a + b;
        a = b; b = c;
    }
    return b;
}
```

O(n) / O(1). *Takeaway:* ways-to-reach-n often reduces to `f(n)=f(n-1)+f(n-2)`.

Q42. House Robber (Medium)

Problem: max sum of non-adjacent elements.

Optimised (DP): at each house, rob it (+ best up to i-2) or skip it (best up to i-1).

```
int rob(vector<int>& a) {
    int prev2 = 0, prev1 = 0;
    for (int x : a) {
        int cur = max(prev1, prev2 + x);
        prev2 = prev1; prev1 = cur;
    }
    return prev1;
}
```

O(n) / O(1). *Takeaway:* "no two adjacent" -> take-or-skip DP.

Q43. Coin Change (Medium)

Problem: fewest coins to make an amount (any coin, unlimited).

Brute force: try all combinations - exponential.

Optimised (DP): `dp[x]` = min coins for amount x.

```cpp
int coinChange(vector<int>& coins, int amount) {
    vector<int> dp(amount + 1, amount + 1);
    dp[0] = 0;
    for (int x = 1; x <= amount; x++)
        for (int c : coins)
            if (c <= x) dp[x] = min(dp[x], dp[x - c] + 1);
    return dp[amount] > amount ? -1 : dp[amount];
}
```

O(amount * coins) / O(amount). *Takeaway:* min/count-of-ways over amounts -> 1D DP.

Q44. Longest Increasing Subsequence (Medium)

Problem: length of the longest strictly increasing subsequence.

Brute force: try all subsequences - exponential.

Optimised (DP O(n^2)): `dp[i]` = LIS ending at i.

```cpp
int lengthOfLIS(vector<int>& a) {
    int n = a.size(), best = 1;
    vector<int> dp(n, 1);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < i; j++)
            if (a[j] < a[i]) {
                dp[i] = max(dp[i], dp[j] + 1);
                best = max(best, dp[i]);
            }
    return n ? best : 0;
}
```

O(n^2) / O(n). *Takeaway:* subsequence DP - `dp[i]` depends on all earlier j. (A binary-search version reaches O(n log n).)

Q45. 0/1 Knapsack (Medium-Hard)

Problem: max value of items fitting in capacity W; each item used once.

Brute force: try every subset - O(2^n).

Optimised (DP): `dp[w]` = best value for capacity w; iterate weight downward so each item is used once.

```cpp
int knapsack(vector<int>& wt, vector<int>& val, int W) {
    vector<int> dp(W + 1, 0);
    for (int i = 0; i < wt.size(); i++)
        for (int w = W; w >= wt[i]; w--)
            dp[w] = max(dp[w], dp[w - wt[i]] + val[i]);
    return dp[W];
```

```
}
```

O(n * W) / O(W). *Takeaway:* the classic "pick items under a limit" DP; downward loop = use each item once.

Q46. Longest Common Subsequence (Medium)

Problem: length of the longest subsequence common to two strings.

Optimised (2D DP): if characters match, +1 on the diagonal; else take the better neighbour.

```
int lcs(string a, string b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            if (a[i - 1] == b[j - 1])
                dp[i][j] = dp[i - 1][j - 1] + 1;
            else
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
    return dp[n][m];
}
```

O(n*m) / O(n*m). *Takeaway:* comparing two sequences -> a 2D grid DP.

Pattern 11: Greedy

Make the locally best choice at each step when that provably leads to a global optimum.

Q47. Best Time to Buy and Sell Stock (Easy)

Problem: max profit from one buy and one later sell.

Brute force: all buy/sell pairs. **O(n^2) / O(1).**

Optimised (greedy): track the lowest price so far, update best profit.

```
int maxProfit(vector<int>& p) {
    int minPrice = INT_MAX, best = 0;
    for (int x : p) {
        minPrice = min(minPrice, x);
        best = max(best, x - minPrice);
    }
    return best;
}
```

O(n) / O(1). *Takeaway:* keep a running best-so-far in one pass.

Q48. Jump Game (Medium)

Problem: can you reach the last index, where each value is a max jump length?

Brute force: try all jump sequences - exponential.

Optimised (greedy reach): track the farthest reachable index.

```
bool canJump(vector<int>& a) {
    int reach = 0;
    for (int i = 0; i < a.size(); i++) {
        if (i > reach) return false;      // stuck
        reach = max(reach, i + a[i]);
    }
    return true;
}
```

O(n) / O(1). *Takeaway:* reachability greedily extends the farthest reach.

Pattern 12: Bit Manipulation

Q49. Single Number (Easy)

Problem: every element appears twice except one; find it.

Brute force: count with a hash map. **O(n) / O(n).**

Optimised (XOR): `x ^ x = 0`, so XOR-ing everything cancels the pairs and leaves the single.

```
int singleNumber(vector<int>& a) {
    int res = 0;
    for (int x : a) res ^= x;
    return res;
}
```

O(n) / O(1). *Takeaway:* XOR cancels duplicates - a classic bit trick.

Pattern 13: Intervals

Sort by start (or end), then sweep and merge/compare neighbours.

Q50. Merge Intervals (Medium)

Problem: merge all overlapping intervals.

Brute force: repeatedly compare and merge pairs. **O(n^2).**

Optimised (sort + sweep): sort by start; if the next overlaps the last, extend it, else start a new one.

```
vector<vector<int>> merge(vector<vector<int>>& v) {
    sort(v.begin(), v.end());
    vector<vector<int>> res;
    for (auto& iv : v) {
        if (!res.empty() && iv[0] <= res.back()[1])
            res.back()[1] = max(res.back()[1], iv[1]);
        else
            res.push_back(iv);
    }
    return res;
}
```

O(n log n) / O(n). *Takeaway:* interval problems almost always start with "sort, then sweep."

The pattern cheat-sheet

When you see a new problem, run down this list - one of them almost always fits:

If the problem is about…	Reach for…
A pair/triplet in a sorted array	Two pointers
A contiguous subarray/substring	Sliding window
"Have I seen this?" / counting	Hash map / set
Many range-sum queries	Prefix sum
Sorted data, or a monotonic yes/no	Binary search
Cycle or midpoint of a linked list	Fast & slow pointers
Matching / nearest greater element	Stack (monotonic)
Anything on a tree	DFS recursion / BFS queue
All subsets/permutations/combinations	Backtracking
Overlapping sub-problems, min/max/count	Dynamic programming
Locally-best choice works	Greedy
Pairs cancel, or bit tricks	XOR / bit manipulation
Overlapping ranges	Sort + sweep intervals

The habit to build: before coding, say out loud which pattern it is and why. That sentence is often half the interview.